

Analysis: Allocation

We want to buy as many as possible houses. Intuitively, we can keep buying the cheapest house. The rationale is to save money at each step so we could buy more in the end. One way to implement this strategy is to sort all the houses by prices from low to high and then buy houses one by one until we run out of money.

The sorting part has $O(N \log N)$ time complexity and the processing part has $O(N)$ time complexity. Using counting sort could reduce the sorting complexity to $O(N)$ since the range of the prices is $[1, 1000]$. The overall time complexity is $O(N)$.

Let's prove the correctness of this greedy algorithm. Let the solution produced by the greedy algorithm be $\mathbf{A} = \{a_1, a_2, \dots, a_k\}$ and an optimal solution $\mathbf{O} = \{o_1, o_2, \dots, o_m\}$.

If $\mathbf{O}$ and $\mathbf{A}$ are the same, we are done with the proof. Let's assume that there is at least one element o_j in $\mathbf{O}$ that is not present in $\mathbf{A}$. Because we always take the smallest element from the original set, we know that any element that is not in $\mathbf{A}$ is greater than or equal to any a_i in $\mathbf{A}$. We could replace o_j with the absent element in $\mathbf{A}$ without worsening the solution, because there will always be element in $\mathbf{A}$ that is not in $\mathbf{O}$. We then increased number of elements in common between $\mathbf{A}$ and $\mathbf{O}$, hence we can repeat this operation only finite number of times. We could repeat this process until all the elements in $\mathbf{O}$ are elements in $\mathbf{A}$. Therefore, $\mathbf{A}$ is as good as any optimal solution.

Analysis: Bundling

We need to maximise the sum of scores of each bundle. Let us consider a bundle and say longest prefix shared by all strings of this bundle is of length X . Now each prefix of length from 1 to X is shared by all of the strings in this bundle. Consider any prefix among these X prefixes, it is counted once in the score of this bundle. Thus the score of a bundle can be defined as number of prefixes that are shared by all of the strings in this bundle. Thus if a prefix is shared by all strings in Y bundles, then it will contribute Y to the total sum of scores.

Now instead of finding the total score, we find the contribution of each prefix in the total score. So for maximising the total score, we would want to maximize the contribution of each prefix in the total score. Let the contribution of each prefix PRE be $contribution(PRE)$. We want to maximize $\sum contribution(PRE)$ where PRE comprises all possible prefixes of the given strings.

Let us say a prefix P_i is a prefix of S strings. The maximum number of bundles of size K formed by S strings is $\lfloor S / K \rfloor$. In each of these $\lfloor S / K \rfloor$ bundles, prefix P_i will add to their scores. Thus maximum value of $contribution(P_i)$ can be $\lfloor S / K \rfloor$. So a prefix P_i which occurs as a prefix of S strings will contribute $\lfloor S / K \rfloor$ to the answer.

Let us see if we can achieve this maximum value for all the prefixes. Consider a prefix P of length L . It occurs as a prefix in CNT number of strings. Now consider there are C prefixes of length $L + 1$ which contain the prefix P as a prefix ($P_1, P_2, \dots, P_C$). And we have stored the number of strings these prefixes are part of as ($CNT_1, CNT_2, \dots, CNT_C$).

Let us say we divided the strings which have prefix P_i into $\lfloor (CNT_i / K) \rfloor$ bundles. Now we have $CNT_i \% K$ strings remaining for each prefix that we need to arrange so that they make a bundle. For each of these remaining strings we cannot have a bundle of size K which would have a common prefix of length $L + 1$ because we have $CNT_i \% K$ remaining strings for each P_i . So, we can make bundles in any order using the remaining strings. Those bundles will still have a prefix of length L shared among them. Thus we would be left with $CNT \% K$ number of strings which are not in any bundle when we consider prefix P . We can continue this procedure till we are left with prefix of length 0. We would be able to bundle all the strings at this point because we would have $N \% K$ strings remaining, and as specified in the problem, N is divisible by K .

The problem is now reduced to finding number of times each prefix occurs in the given strings. Let this number be $COUNT$. We just need to add $\lfloor COUNT / K \rfloor$ to the answer for each prefix.

Test set 1

The length of each string is at most 5. Thus we have total number of prefixes as $N * 5$ and each prefix can be of size at most 5. Store each prefix in a hashmap and increase the count for each prefix. In the end, we just need to add $\lfloor (count(i) / K) \rfloor$ for each prefix i . The complexity of the solution would be $O(N * 5 * 5)$.

Test set 2

Let the sum of length of all strings over all the test cases be SUM which is 10^6 . For the large test set, the length of the string can be very large. So, we can't store all the prefixes in a hashmap. We need to store all the prefixes in an efficient manner along with the number of times they occur in given strings. We can use a data structure [trie](#). The insertion cost would be

equal to sum of length of strings over the test cases which is $O(\text{SUM})$. Then finally we just need to traverse the trie and for each prefix, add its contribution to the answer. Time complexity of the solution would be $O(\text{SUM})$.

Kick Start 2020 - Round A

Analysis: Plates

From the constraints, we can see that regardless of the test set, $1 \leq K \leq 100$. i.e., $1 \leq P \leq 100 \cdot N$.

Test set 1

For this test set, we see that $1 \leq N \leq 3$. So, we can check for every possible combination of taken plates across all stacks and output the maximum sum. For example, if $N = 3$ and for any given values of K and P , generate all possible triples (S_1, S_2, S_3) such that $S_1 + S_2 + S_3 = P$ and $0 \leq S_i \leq K$. Note: S_i is the number of plates picked from the i -th stack.

This can be done via recursion and the total time complexity is $O(K^N)$ which abides by the time limits.

Test set 2

The solution we had for test set 1 doesn't scale given that N now is at most 100. In order to tackle this test set, we use Dynamic Programming along with some precomputation.

First, let's consider an intermediate state $dp[i][j]$ which denotes the maximum sum that can be obtained using the first i stacks when we need to pick j plates in total. Therefore, $dp[N][P]$ would give us the maximum sum using the first N stacks if we need to pick P plates in total. In order to compute $dp[i][j]$ efficiently, we need to be able to efficiently answer the question: *What is the sum of the first x plates from stack i ?* We can precompute this once for all N stacks. Let $sum[i][x]$ denote the sum of first x plates from stack i .

Next, we iterate over the stacks and try to answer the question: *What is the maximum sum if we had to pick j plates in total using the i stacks we've seen so far?* This would give us $dp[i][j]$. However, we need to also decide, *among those j plates, how many come from the i -th stack?* i.e., Let's say we pick x plates from the i -th stack, then $dp[i][j] = \max(dp[i][j], sum[i][x] + dp[i-1][j-x])$. Therefore, in order to pick j plates in total from i stacks, we can pick anywhere between $[0, 1, \dots, j]$ plates from the i -th stack and $[j, j-1, \dots, 0]$ plates from the previous $i-1$ stacks respectively. Also, we need to do this for all values of $1 \leq j \leq P$.

The flow would look like:

```
for i [1, N]:
  for j [0, P]:
    dp[i][j] := 0
    for x [0, min(j, K)]:
      dp[i][j] = max(dp[i][j], sum[i][x] + dp[i-1][j-x])
```

If we observe closely, this is similar to the [0-1 Knapsack Problem](#) with some added complexity. To conclude, the overall time complexity would be $O(N \cdot P \cdot K)$.

Analysis: Workout

Test set 1

Since $K=1$, all that we need to do is to find the maximum difference and split it into 2 halves. For example, given a sequence $[2, 12, 18]$ and $K = 1$, the *difficulty* is 10, since the maximum difference is in $[2, 12]$. The best way to minimize this is to take the maximum difference and split it in half giving us the final sequence of $[2, 7, 12, 18]$. The *difficulty* for this final sequence now is 6. The time complexity is $O(N)$.

Test set 2

For this test case, we cannot perform such direct splits because repeatedly splitting the maximum difference into halves is not optimal. For example, given a sequence $[2, 12]$ and $K = 2$, splitting into halves will result in $[2, 12] \rightarrow [2, 7, 12] \rightarrow [2, 7, 9, 12]$. This way, the *difficulty* would be 5. However, if we perform $[2, 12] \rightarrow [2, 5, 12] \rightarrow [2, 5, 8, 12]$, the *difficulty* would be 4. This clearly demonstrates that continuous halving of the maximum difference is sub-optimal. Okay, so how do we do this?

Consider the i -th adjacent pair of training sessions with an initial difference d_i . If we want to insert some number of training sessions in between this pair such that the maximum difference among those is at most a certain value, let's say d_{optimal} , then *how many training sessions can be inserted in between?* The answer to this is $\text{ceiling}(d_i / d_{\text{optimal}}) - 1$. Let's call that k'_i . Doing this for all $N-1$ adjacent pairs in the given array would give us $k'[1, \dots, N-1]$. Let's denote $k'_{\text{sum}} = k'_1 + k'_2 + \dots + k'_{N-1}$. From the constraints, we can insert at most K training sessions. Therefore, we need to make sure $k'_{\text{sum}} \leq K$ while minimizing d_{optimal} as much as possible.

If you observe, d_{optimal} can lie anywhere between $[1, \max(d_i)]$ ($1 \leq i \leq N-1$). Linear search would be to check every value here starting from 1 and output the first value that satisfies the above condition. A quicker way to do this is using binary search. On closer observation, you can see that increasing the value of d_{optimal} decreases the value of $\text{ceiling}(d_i / d_{\text{optimal}}) - 1$ and hence smaller is the value of k'_{sum} . Therefore, we can perform a binary search in the range $[1, \max(d_i)]$ to find the least value of d_{optimal} that makes $k'_{\text{sum}} \leq K$. That is our answer.

Since the $\max(d_i)$ could be as much as 10^9 , we might have to search $[1, 10^9]$ making time complexity of the solution is $O(\log(10^9) * N)$.